RabbitMQ

Assignment

Problem Statement:

In modern distributed architectures, RabbitMQ is widely used to decouple services, buffer heavy workloads, and route messages efficiently.

For this assignment, you are required to build an **E-Commerce Order Processing System** that demonstrates asynchronous communication and advanced message routing using a Topic Exchange in RabbitMQ. When a user places an order, multiple independent background services should react to the event based on specific routing rules.

The Architecture & Flow:

This assignment requires both a Frontend (UI) and a Backend (API server and Consumer Workers).

- **Frontend (UI):** A simple page with a form to place an order, selecting an item category (e.g., "electronics" or "clothing") and a quantity. When submitted, it sends an API request to the backend.
- **Backend Publisher (Order Service):** An API endpoint (e.g., /api/orders) that receives the request, publishes an order_placed event to a Topic Exchange (named `orders_exchange`) with a routing key format like `order.created.<category>` (e.g., order.created.electronics). The message should include a unique order_id in its properties. It **immediately** responds to the frontend with a success message.
- **Backend Subscriber 1 (Inventory Service):** A consumer worker that binds its queue to the orders_exchange with the binding key `order.created.*`. It listens for all orders and simulates reserving inventory.
  Crucial feature: If the quantity is invalid (e.g., < 0), it must explicitly reject the message without requeuing.
- **Backend Subscriber 2 (Specialized Analytics Service):** A consumer worker that binds its queue to the orders_exchange with the binding key `order.created.electronics`. It only processes electronics orders and ignores other categories. It simulates logging analytics by printing: `"Analytics Service: Recording high-value electronics sale for order [Order ID]"`.
- **Backend Subscriber 3 (Error Handling via DLX):** A consumer that listens to a Dead Letter Queue. When the Inventory Service rejects an invalid order, the broker should route it to a Dead Letter Exchange, where this service logs it: `"Error Logger: Invalid order [Order ID] routed to DLX"`.

Technology Requirement:

- **Backend:** You can choose any programming language and framework (e.g., Node.js, Python, Java, Go). You must use RabbitMQ as your message broker.
- **Implementation Details:**
  - You MUST implement Manual Acknowledgments (`autoAck = false` or equivalent) in your consumers to acknowledge the message only after successful processing.

    - You MUST use a Topic Exchange for routing to demonstrate wildcard and exact match routing.
    - You MUST configure Fair Dispatch by setting the Prefetch Count (`prefetch_count = 1`) on your consumer channels.
    - You MUST demonstrate Idempotent Design: Consumers should check the unique `order_id` (simulated via an in-memory set or map) to ensure they don't process the same order twice.
    - You MUST implement a Dead Letter Exchange (DLX) for handling rejected messages.
- **Frontend (Preferable but Optional):** Building a frontend is recommended to get a complete understanding of the flow. However, if you prefer not to build a UI, you can use Postman, cURL, or a CLI tool to trigger the backend API.

Deliverables:

- **Code Repository:** Upload the working code to a GitHub repository and MUST share the PR link and branch name. The repository must include the Publisher API and both Consumer services.
- **Documentation** (README.md): Provide clear instructions on how to start RabbitMQ (e.g., providing the Docker run command), how to run your application components, and how to trigger the API.
- **Execution Proof:** Include screenshots or a short video clip showing the API request being made, and both Subscribers processing the event accurately based on the routing key (e.g., showing both services receiving an "electronics" order, but only the Inventory service receiving a "clothing" order).
- **Status Updates:** Please keep sharing daily status updates with your assignment progress.